

IES LOS SAUCES
BENAVENTE (ZAMORA)



GRADO SUPERIOR DE
DESARROLLO DE APLICACIONES WEB

EXPORTADOR DATOS DE VIDEO LARAVEL

DEPARTAMENTO: INFORMÁTICA



AUTOR: Alejandro De la Huerga Fernández

CURSO: 2025/26

PERIODO: JUNIO

TUTORÍA COLECTIVA: Hermelinda Ramos Osorio

TUTORÍA INDIVIDUAL: Heraclio Borbujo Moran

LARAVEL

EXPORTADOR DE DATOS DE VIDEO



Alejandro De la Huerga Fernández

TABLA DE CONTENIDO

INTRODUCCIÓN	3
Estructura	3
Objetivos del Proyecto	3
¿Por qué Laravel?	3
ESTUDIO TEÓRICO LARAVEL Y ARCHIVOS DEMO	7
Características Laravel	8
Estructura de archivos laravel.....	9
Estructura de directorios (teórico – práctico)	12
Flujo de trabajo.....	14
Eloquent ORM	15
Archivos Demo	16
¿Qué son los archivos demo?	16
Diferencias archivos demo y archivos .dem	16
Donde Obtener los Archivos Demo	18
ANÁLISIS APLICACIÓN DEMOSCAN	22
Catálogo de requisitos.....	22
Diagrama de casos de uso	27
Modelo Físico de Datos	30
Tablas de la base de datos	30
DISEÑO Y DESARROLLO	37
Preparación Entorno de desarrollo	37
Entorno desarrollo local (Laragon)	37
Desarrollo de la aplicación	42
IMPLANTACIÓN DE LA APLICACIÓN.....	50
WEBGRÁFIAS Y REFERENCIAS	55
CONCLUSIONES	55

INTRODUCCIÓN

El siguiente documento es una guía completa sobre el estudio, planificación y diseño del proyecto final desarrollado para el ciclo de grado superior de desarrollo de aplicaciones web.

Este documento tiene como objetivo informar y guiar a los usuarios desde el estudio del **framework Laravel** el cual ha sido elegido marco de trabajo para desarrollar la aplicación, así como el proceso de diseño y estudio de la aplicación de principio a fin.

ESTRUCTURA

Durante este documento se seguirá una estructura de explicación y desarrollo mediante la cual se explicará en primer lugar aspectos más generales como el estudio del framework así como su estructura de directorios o forma de trabajo, como los aspectos más individuales o específicos como toda la información a tener en cuenta sobre los archivos **demo**.

Se concluirá el estudio con la explicación y muestra de la aplicación web con la funcionalidad de extracción de datos.

OBJETIVOS DEL PROYECTO

El proyecto tiene como objetivo central la explicación detallada y completa del framework de desarrollo **Laravel**.

Mediante este estudio se busca recoger y explicar cada uno de los aspectos importantes y funcionalidades que tiene este framework para el desarrollo de aplicaciones web.

Los objetivos del desarrollo del proyecto son los siguientes:

1. *Análisis y diseño de la aplicación de extracción de datos de video de Laravel.*
2. *Estudio teórico sobre Laravel y sus características.*
3. *Preparación del entorno de desarrollo.*
4. *Desarrollo y prueba del proyecto Aplicación de extracción de datos de video.*
5. *Implantación del trabajo (producción).*

¿POR QUÉ LARAVEL?

La elección de este framework para desarrollar nuestra aplicación web, es debido a varios factores los cuales analizamos previamente y comparamos Laravel con otras opciones posibles, a continuación, el resultado del análisis:

COMPARATIVA CON OTROS FRAMEWORKS

A continuación, vamos a ver una comparativa en la cual compararemos las diferentes características de Laravel con otros frameworks web.

Para ello vamos a realizar comparaciones uno a uno para que de esa forma se puedan distinguir bien las diferencias de ambos frameworks.

LARAVEL Y DJANGO

Ambos frameworks son un **todo incluido**, es decir traen todo un sistema de gestión de base de datos, autenticación y rutas.

Las características que hacen destacar a uno por encima del otro son las siguientes:

- **Laravel:** Sintaxis elegante y expresiva, además su sistema Eloquent ORM esta muy por encima del sistema de Django.
- **Django:** Fácil integración si la aplicación web va a integrar Inteligencia Artificial o algún módulo de Machine Learning.

Carácterística	Laravel (PHP)	Django (Python)
Arquitectura	MVC (Basado en componentes)	MVT (Model-View-Template)
Curva de aprendizaje	Baja - Media	Media
ORM	Eloquent (Activo y potente)	Django ORM (Robusto)
Velocidad de desarrollo	Muy rápida	Muy rápida
Ecosistema Frontend	Excelente (Vite, Blade, Vue/React)	Bueno (Templates propios)

LARAVEL Y EXPRESS.JS

Estos dos frameworks son muy similares ya que ambos son utilizados por los desarrolladores cuando se busca escalabilidad en tiempo real.

Las características que hacen destacar a uno por encima del otro son las siguientes:

- **Laravel:** Te da una estructura clara. En Express, cada desarrollador puede organizar el proyecto como quiera, lo que a menudo genera "código espagueti" en proyectos grandes si no hay un equipo senior.
- **Expres.js:** El rendimiento bruto en aplicaciones con miles de conexiones simultáneas (como un chat o streaming) es mayor debido al bucle de eventos no bloqueante de Node.js.

Característica	Laravel (PHP)	Express.js (Node.js)
Arquitectura	MVC (Basado en componentes)	Minimalista
Curva de aprendizaje	Baja - Media	Muy baja
ORM	Eloquent (Activo y potente)	Sequelize / Prisma (Externos)
Velocidad de desarrollo	Muy rápida	Media (Requiere configurar todo)
Ecosistema Frontend	Excelente (Vite, Blade, Vue/React)	Total (React, Vue, Angular)

LARAVEL Y SPRING BOOT

Estos dos frameworks presentan dos arquitecturas de software totalmente diferentes, mientras que Laravel destaca por su agilidad y elegancia, Spring Boot se posiciona por encima del resto de frameworks en cuanto a la robustez y arquitectura estricta.

Las características que hacen destacar a uno por encima del otro son las siguientes:

- **Laravel:** proporciona una estructura sólida y decisiones predeterminadas sobre cómo hacer las cosas (opinado), pero te permite modificar o reemplazar esas reglas cuando tu proyecto lo requiera (flexible).
- **Spring Boot:** Está diseñado bajo la filosofía de Inyección de Dependencias (DI) y Programación Orientada a Aspectos (AOP). Es extremadamente modular. No es un framework "todo en uno" por defecto, sino que tú añades "starters" según lo que necesites.

Carácterística	Laravel (PHP)	Spring Boot (Java)
Arquitectura	MVC (Basado en componentes)	Basado en microservicios
Curva de aprendizaje	Baja - Media	Alta
ORM	Eloquent (Activo y potente)	Hibernate (Complejo)
Velocidad de desarrollo	Muy rápida	Lenta / Estrcuturada
Ecosistema Frontend	Excelente (Vite, Blade, Vue/React)	Limitado

ESTUDIO TEÓRICO LARAVEL Y ARCHIVOS DEMO

Laravel es un framework desarrollado y basado en el **lenguaje de programación PHP**, el cual nos permite el desarrollo e implementación de aplicaciones web seguras de manera **rápida e intuitiva**.

Laravel nos permite el desarrollo de aplicaciones web escalables a lo largo del tiempo de manera **óptima**.

Documentación oficial: <https://laravel.com/docs/12.x/installation>

¿POR QUÉ LARAVEL PARA LA APLICACIÓN FINAL?

La elección del desarrollo de la aplicación web capaz de extraer la información de los archivos demos es debido a que Laravel nos permite la construcción de una estructura de **Login Log off** y estructura de autenticación en la base de datos de manera muy rápida justo antes de crear el propio proyecto.

Esto nos permitirá **centrarnos en la funcionalidad de la aplicación desde el primer momento**, optimizando el trabajo y sin preocuparnos por las funcionalidades más comunes de las aplicaciones web.



Además, nos permite la creación y administración de la base de datos mediante **Eloquent ORM** y su sistema de migraciones que veremos más a fondo más adelante.

CARACTERISTICAS LARAVEL

Laravel es uno de los frameworks de desarrollo de aplicaciones web más utilizados por sus desarrolladores, esta elección por parte de los profesionales del sector es debido a todo su ecosistema y sus grandes características en cuanto al desarrollo web se refiere:

- **Seguridad Integrada:** Laravel gestiona automáticamente la seguridad, incluyendo protección contra inyecciones SQL, falsificación de peticiones entre sitios.
- **Gestión de Dependencias (Composer):** Utiliza Composer para manejar bibliotecas de terceros y paquetes, lo que facilita la actualización y gestión del proyecto.
- **Migraciones de Base de Datos:** Permite gestionar la estructura de la base de datos mediante código PHP (control de versiones), facilitando la actualización y compartición de la estructura sin usar SQL directo.
- **Artisan:** Una interfaz de línea de comandos integrada que automatiza tareas repetitivas de desarrollo, como la creación de migraciones, controladores y modelos.
- **Eloquent ORM:** Es una implementación avanzada de ActiveRecord que permite interactuar con la base de datos utilizando sintaxis PHP en lugar de SQL, facilitando las consultas y la manipulación de datos.
- **Motor de Plantillas Blade:** Un motor rápido y potente que permite usar código PHP en vistas, estructura jerárquica de plantillas y reutilización de datos, mejorando la visualización.

VENTAJAS DEL DESARROLLO CON LARAVEL

Laravel cuenta con un gran número de ventajas que lo sitúan por encima del resto de frameworks del mercado en cuanto al desarrollo web se refiere:

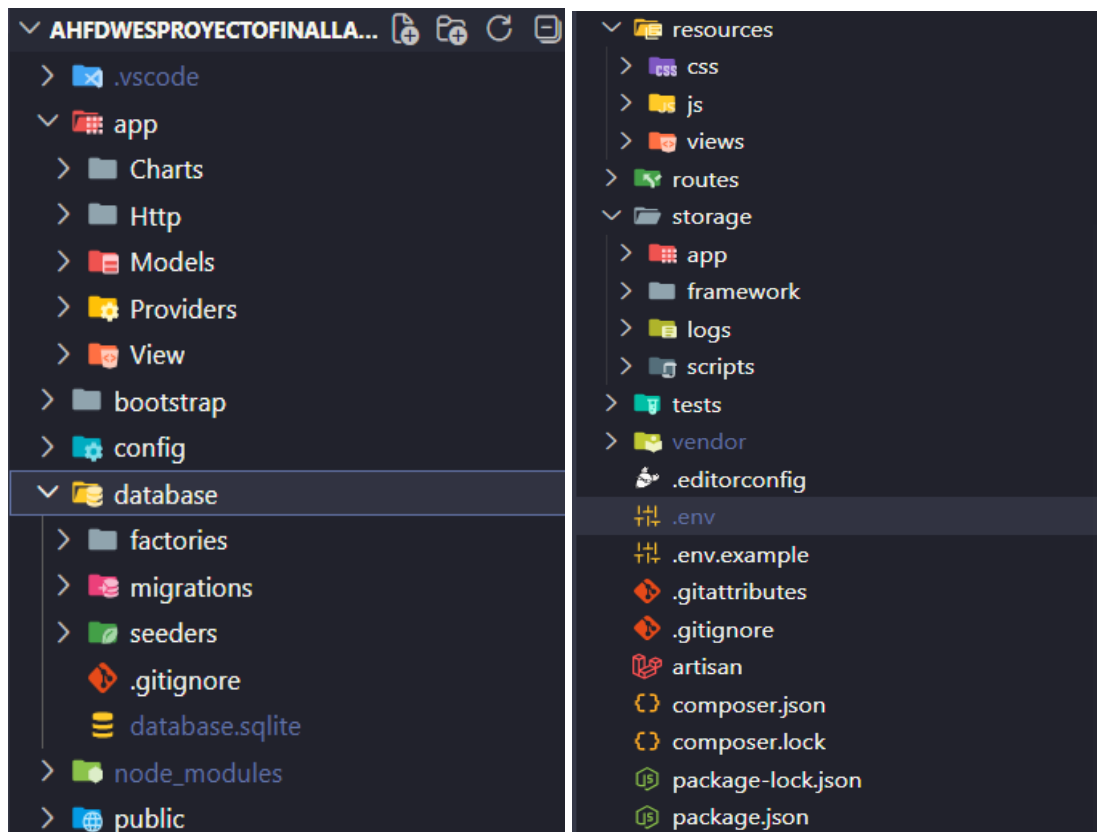
- **Desarrollo Rápido y Eficiente:** Su sintaxis elegante y herramientas predefinidas automatizan tareas comunes, permitiendo crear aplicaciones web complejas en menos tiempo.
- **Sintaxis y estructura organizada:** Una de las principales ventajas de Laravel es su elegante sintaxis que facilita la lectura y el mantenimiento del código. Con una estructura organizada y clara, el desarrollo de aplicaciones se vuelve más eficiente y menos propenso a errores.

- **Ciberseguridad:** Trae opciones de configuración orientadas a conservar la seguridad de accesos. El framework se encarga de verificar la identidad del usuario que quiera manipular el código pidiendo la contraseña en múltiples ocasiones.

ESTRUCTURA DE ARCHIVOS LARAVEL

Al crear nuestra aplicación Laravel se nos crea una **estructura de directorios común de cada proyecto Laravel**, dentro de esta estructura podemos diferenciar los diferentes tipos de archivos según su funcionalidad.

Vamos a realizar un estudio de esta estructura junto con la explicación de la funcionalidad de cada directorio:



- **app/:** Aquí se encuentra toda la lógica de nuestra aplicación.
 - **Models/:** Contiene las clases que representan las tablas en la base de datos (Eloquent ORM).
 - **Http/Controllers:** Aquí se encuentran todos los controladores de la aplicación.
 - **Http/Middleware:** Filtros para controlar las peticiones.

- **config/**: Contiene los archivos de configuración para la base de datos, caché, servicios de correo, etc.
- **routes/**: Define los puntos de entrada a nuestra aplicación.
 - **web.php**: Aquí van las rutas para nuestro navegador.
 - **api.php**: Rutas para servicios externos o apps móviles.
- **database/**: Aquí es donde se encuentra la capa de datos. Dentro podemos encontrar diferentes directorios:
 - **migrations/**: Son aquellos archivos los cuales crean las tablas en nuestra base de datos, funcionan como un control de versiones:

```
public function up(): void
{
    // Creación de la tbla en la base de datos analisis.
    Schema::create('analisis', function (Blueprint $table) {
        $table->id();
        $table->foreignId('demo_id')->constrained('demos')->cascadeOnDelete();
        $table->json('resultado_json');
        $table->string('nombre_mapa');
        $table->integer('rondas_jugadas');
        $table->integer('puntuacion_equipo1');
        $table->integer('puntuacion_equipo2');
        $table->timestamp('fecha_analisis');
    });
}

/**
 * Reverse the migrations.
 * Función la cual realiza n rollback y vuelve atras las migraciones antes de ejecutar el up.
 */
public function down(): void
{
    // Borra la tabla análisis
    Schema::dropIfExists('analisis');
}
};
```

El método `up()` es el que crea la tabla en la base de datos y el `down()` la función que haría el rollback.

Comando de ejecución: **php artisan migrate** (Este comando ejecuta todas las migraciones que tengamos en la carpeta `migrations`).

- **seeders/**: Archivos los cuales rellenan las tablas de la base de datos para pruebas iniciales.

```
Usuario::insert([
    [
        'nombre' => 'Administrador',
        'correo' => 'alejandrohurga.dev@gmail.com',
        'password' => Hash::make('paso1234')
    ],
    [
        'nombre' => 'Administrador2',
        'correo' => 'whoishuergale@gmail.com',
        'password' => Hash::make('paso1234')
    ]
]);
}
```

Comando de ejecución: **php artisan db:seed**

- **factories/**: Generadores de datos aleatorios para pruebas masivas.
- **resources/**: Aquí es donde se encuentra toda la capa visual de nuestra aplicación.
 - **Views/**: Archivos Blade (Motor de plantillas de Laravel), los cuales representan las vistas de nuestra aplicación y tienen la extensión .blade.php.
 - **css/ y js/**: Archivos de estilos y scripts antes de ser procesados.
- **public/**: Este es el único directorio al cual el servidor debe de tener acceso contiene el index.php, imágenes que utilice nuestra aplicación, CSS y JS ya compilados.
- **storage/**: Donde Laravel guarda archivos temporales, logs de errores y archivos subidos por usuarios (En la primera versión de nuestra aplicación se almacenaba aquí el archivo .dem). Ahora **almacenamos los scripts que extraen la información del archivo** junto con las librerías.
- **vendor/**: Librerías de terceros que hemos instalado mediante composer.
- **Archivos importantes de configuración en la raíz:**
 - **.env**: Configuración de variables de entorno (credenciales de BD, claves secretas). Es sensible y no se sube a GitHub.
 - **composer.json**: Lista de dependencias php que nuestro proyecto necesita para funcionar.
 - **vite.config.js**: Configura el compilador de archivos del cliente.

ESTRUCTURA DE DIRECTORIOS (TEÓRICO – PRÁCTICO)

A continuación, vamos a analizar la estructura de directorios de un proyecto Laravel desde un punto de vista teórico práctico (para que sirva).

- **app:** Es el directorio mas importante del proyecto, dentro de el se encuentra toda la lógica de negocio (controladores, modelos, middleware) es el corazón de la aplicación.
 - **Http/Controllers:** Aquí se encuentran los controladores de nuestra aplicación, su principal función es conectar la parte visual de la aplicación con la lógica de la aplicación (Modelo, Base de datos). Reciben Request y devuelven respuestas.
 - **Models:** Aquí se encuentran todos los modelos de la aplicación, su principal función es la de consultar y gestionar datos en la base de datos.
- **Config:** Este directorio tiene como función principal almacenar todos los archivos de configuración de la aplicación, estos archivos tienen en su interior diferentes parámetros de configuración, muchos de ellos luego se utilizan en el archivo .env
- **Database:** Directorio que almacena los diferentes directorios relacionados con la base de datos.
 - **Factories:** Aquí se encuentran los archivos de Factories cuya función es la del ingreso de datos falsos a la base de datos de manera automática para pruebas.
 - **Migrations:** Aquí se encuentran todas las migraciones, cuya función es la de crear un versionado estructural de la base de datos, pudiendo así retroceder y avanzar a lo largo del ciclo de vida de la base de datos.
 - **Seeders:** Su principal función es la de crear datos fake/Iniciales, se ejecutara al crear la base de datos por mediación de las migraciones y la creara con los datos establecidos en el seeder desde el principio.
- **Public:** Es la única carpeta pública de todo el proyecto Laravel, todo lo que incluya este directorio será lo que sea capaz de ver el usuario en su pantalla, aquí si tiene acceso la vista.
- **Resources:** Su función es almacenar todos aquellos archivos que no han sido compilados en nuestra aplicación.
- **Routes:** Este directorio almacena todos los archivos con las rutas de nuestra aplicación, su principal función es conectar aquellas url que solicita el usuario ya sea mediante la pulsación de un botón o

enviando un formulario y conectarlas con un controlador el cual realiza un método o acción devolviendo una respuesta.

- **Storage:** Este directorio es el encargado de almacenar todos aquellos archivos generados por nuestra aplicación o almacenar librerías externas que vamos a utilizar en nuestra aplicación.

TABLA RELACIONAL ARCHIVOS CON APLICACIÓN DEMOSCAN

La siguiente tabla relaciona los directorios de estructura de Laravel con un ejemplo en nuestra aplicación **DemoScan** desarrollada.

DIRECTORIO DE LARAVEL	ARCHIVO APLICACIÓN DEMOSCAN
Models	Jugador.php (Modelo de nuestra aplicación que representa a un Jugador de Counter Strike).
Controllers	DemoController.php (Controlador el cual tiene el método guardarArchivo() cuya funcionalidad es guardar el archivo demo en la base de datos en formato Json).
Routes	web.php (Archivo en el cual se encuentran todas las rutas de nuestra aplicación, como por ejemplo la ruta de acceso a las páginas mediante las pestañas del nav).
Migrations	2026_03_16_134546_analisis.php (Archivo el cual tiene los métodos up() y down() que crean o borran la tabla analises en la base de datos).
Seeders	UsuariosSeeders.php (Archivo el cual contiene el método insert y que inserta usuarios en la base de datos al crearla).
Views	Dashboard.blade.php (Archivo que contiene la vista donde se importa el archivo demo de nuestra aplicación).
Storage/scripts	Parse.cjs (Archivo javascript con toda la lógica para explotar el archivo demo y sacar la información en JSON).

Esta tabla muestra ejemplos de archivos y su funcionalidad de la aplicación DemoScan, solamente se ha ejemplificado aquellos ejemplos mediante los cuales se ha desarrollado la aplicación.

FLUJO DE TRABAJO

Laravel sigue un flujo de trabajo claro y estructurado al ser un Framework que trabaja con la estructura **Modelo – Vista – Controlador (MVC)**:

1. Llega la petición a *routes/web.php*, por ejemplo un inicio de sesión.

```
Route::get('/analisis', [App\Http\Controllers\AnalisisController::class,
'seleccionarPartidosUsuario'])
->middleware(['auth', 'verified'])->name('analisis');
```

Ejemplo de una ruta que llama al método "seleccionarPartidosUsuario" del controlador AnalisisController solo para usuarios que hayan iniciado sesión mediante el método del middleware.

2. La ruta llama al método de un controlador como vemos en la imagen anterior.
3. El controlador consulta el Modelo:

```
/**
 * Muestra la vista principal con los análisis del usuario.
 */
public function seleccionarPartidosUsuario()
{
    $analisisSubidos = Analisis::where('user_id', Auth::id())->latest()->get();
    return view('analisis', compact('analisisSubidos'));
}
```

Método del controlador AnalisisController el cual llama al modelo Analisis y devuelve una vista.

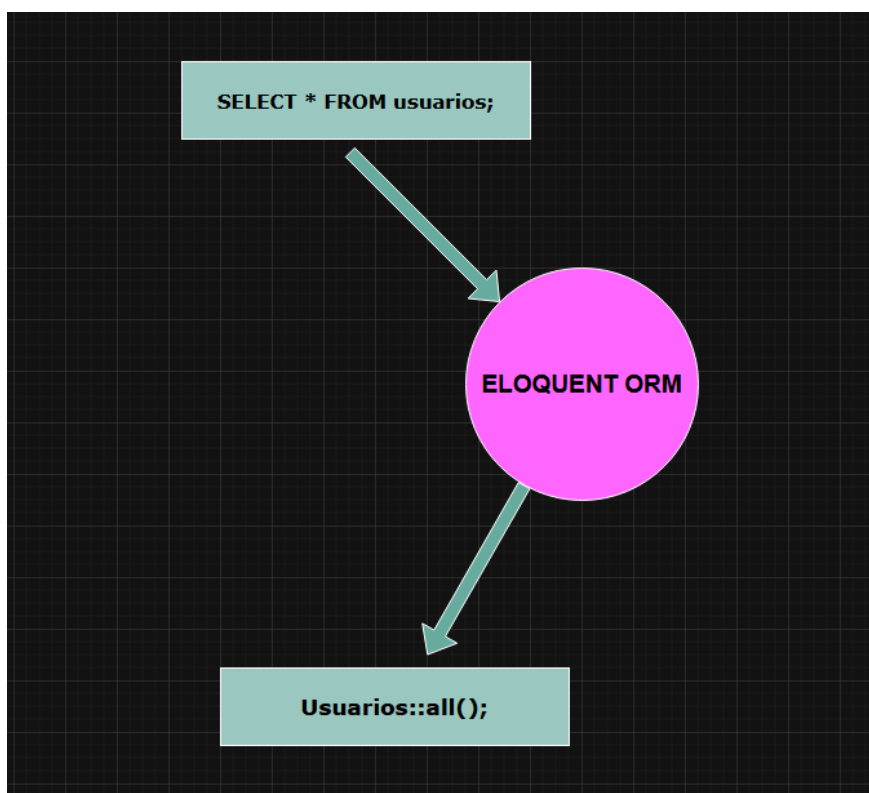
ELOQUENT ORM

Eloquent ORM es el mapeador objeto-relacional de Laravel el cual nos permite

interactuar con la base de datos utilizando funciones PHP expresivas y orientadas a

objetos elegantes en lugar de la sintaxis tradicional de SQL (**SELECT * FROM usuarios**).

Para una mejor explicación de Eloquent ORM, voy a representarlo mediante un esquema:



Como podemos ver en el esquema Eloquent nos permite cambiar la **sintaxis SQL** utilizada normalmente para hacer peticiones a la base de datos por una **sintaxis PHP orientada a objetos** y más simple.

Documentación oficial Eloquent ORM:

<https://laravel.com/docs/12.x/eloquent>

ARCHIVOS DEMO

Antes de continuar con el estudio de nuestra aplicación necesitamos saber que son y toda la información necesaria para poder trabajar con ellos de manera eficiente y segura.

¿QUÉ SON LOS ARCHIVOS DEMO?

Los archivos demo, son archivos de demostración comúnmente utilizados y generados por los motores de videojuegos, aunque hace años se empezaron a implementar en servicios de realidad virtual y en cámaras con módulos de Inteligencia Artificial integrados.

Los archivos demo desempeñan la función de almacenar información de repetición de manera continua.

Es decir, si jugamos una partida al videojuego Counter Strike, una vez terminada la partida ese archivo contiene todo lo ocurrido en cada instante desde la perspectiva de cada uno de los 10 jugadores que forman los equipos.

DIFERENCIAS ARCHIVOS DEMO Y ARCHIVOS .DEM

Antes de avanzar con el análisis de los archivos y sus funcionalidades debemos de saber diferenciar entre dos tipos de archivos cuyos dos llevan la misma extensión (.dem) pero que están muy lejos de ser el mismo tipo de archivo, sus usos o su estructura lo cual puede llevar a confusión.

ARCHIVOS DEMO (COUNTER STRIKE 2)

1. **Qué es:** Son archivos de repetición, almacenan toda la información de una partida disputada en el videojuego desde la perspectiva del jugador.
2. **Contenido:** No se trata de un video tradicional, sino un registro de movimientos y eventos de todo aquello que ocurre durante el transcurso de una partida.
3. **Uso:** Se utilizan sobre todo en competiciones profesionales para el estudio y análisis de rivales, así como la implementación de estrategias o para poder identificar aquellos usuarios que hacen trampas.
4. **Como abrirlo:** No se abren con programas externos, para poder reproducirlos es necesario hacerlo desde dentro del video juego para así utilizar el motor de este. Dentro del juego podemos utilizar la consola para reproducirlos utilizando el comando *playdemo*.

ARCHIVOS DEM (TOPOGRAFÍA Y GEOGRAFÍA)

1. **Qué es:** Significa *Modelo Digital de Elevación*, es un tipo de archivo de mapa en formato raster como por ejemplo un archivo GeoTIFF,.asc o .hgt .
2. **Contenido:** Cada píxel de archivo guarda la información sobre la elevación del terreno en ese píxel sobre el nivel del mar.
3. **Uso:** Se utilizan en Sistemas de Información Geográfica (SIG), como QSIG o ArcGIS, para crear curvas de nivel, modelado 3D de paisajes o estudios hidrológicos.
4. **Como abrirlo:** Se pueden abrir o importar directamente en programas especializados en la visualización de mapas como GSINS o Global Mapper.

ESTRUCTURA Y CONTENIDO INTERNO DE UN ARCHIVO DEMO (CS2)

Dirección	Representación Hexadecimal	Texto ASCII
0x00000000	50 42 44 45 4D 53 32 00 BE 76 A8 27 35 6B A8 27	PBDEMS2..v.'5k.'
0x00000010	01 FF FF FF FF 0F 90 01 0A 08 50 42 44 45 4D 53	PBDEMS
0x00000020	32 00 10 BD 6E 1A 0B 44 72 61 63 75 4C 41 4E 20	2...n..DracuLAN
0x00000030	53 34 22 0D 53 6F 75 72 63 65 54 56 20 44 65 6D	S4".SourceTV Dem
0x00000040	6F 2A 07 64 65 5F 6E 75 6B 65 32 19 2F 68 6F 6D	o*.de_nuke2./hom
0x00000050	65 2F 63 6F 6E 74 61 69 6E 65 72 2F 67 61 6D 65	e/container/game
0x00000060	2F 63 73 67 6F 38 02 40 01 48 01 52 00 5A 0C 76	/csgo8.@.H.R.Z.v
0x00000070	61 6C 76 65 5F 64 65 6D 6F 5F 32 62 24 38 65 39	alve_demo_2b\$8e9
0x00000080	64 37 31 61 62 2D 30 34 61 31 2D 34 63 30 31 2D	d71ab-04a1-4c01-
0x00000090	62 62 36 31 2D 61 63 66 65 64 65 32 37 63 30 34	bb61-acfede27c04
0x000000A0	36 68 B8 53 78 AD D1 03 08 FF FF FF FF 0F 08 1A	6h.Sx.....
0x000000B0	06 04 01 02 6B F4 00 08 FF FF FF FF 0F 0E 1A 0C	...k.....
0x000000C0	D3 24 28 1C 90 95 7D B9 D5 AD 95 05 08 FF FF FF	.\$(...)}.....
0x000000D0	FF 0F EB B2 01 1A E7 B2 01 9C 8C 28 3C 9C 95 B9	(<...
0x000000E0	95 C9 A5 8D C1 C9 95 8D 85 8D A1 95 41 04 60 04	A.`.
0x000000F0	80 04 A0 08 C0 00 E8 08 0C 20 20 01 40 05 70 B2	@.p.
0x00000100	FE A4 00 91 E6 36 47 17 E6 36 56 26 16 36 57 C6	6G..6V&.6W.
0x00000110	96 E6 56 06 31 83 01 00 02 80 02 00 13 A0 53 FC	..V.1.....S.
0x00000120	A4 AF 05 EF 33 1B 19 80 2C 01 00 02 00 10 FC 01	3.....
0x00000130	A0 F0 08 40 E1 01 00 00 09 02 00 00 0D 02 00 00	..@.....
0x00000140	05 A2 00 00 00 00 00 58 82 A0 FE 7B B0 6E 03 D8	X...{.n..
0x00000150	4C 0E 40 96 00 00 12 00 00 00 A0 2A 92 92 7A 92	L.@.....*.Z.
0x00000160	4A 9A A2 12 10 42 03 00 70 54 E6 46 C7 56 06 D2	J...B...pT.F.V..
0x00000170	14 46 57 36 17 20 81 0A AC 86 06 20 EB 26 1B 0B	.FW6.&..
0x00000180	80 79 63 67 0E 00 00 00 FD FF FF 1F 04 F1 00 B1	.ycg.....
0x00000190	63 3C 15 55 38 FF FF FF 07 00 00 00 FC 01 00 00	<<.U8.....
0x000001A0	18 00 10 F0 02 22 F8 03 00 F0 FF FF 7F 00 50 F0	".P.
0x000001B0	40 F4 47 00 00 00 80 00 00 A8 61 23 00 00 E0 21	@.G.....a#!
0x000001C0	00 00 00 50 70 88 0C 80 BB 45 00 00 96 C3 00 00	...Pp....E.....
0x000001D0	00 40 CD CC 4C 3E 01 00 00 FE 10 68 27 3A 96 47	...@..L>....h':.G
0x000001E0	13 5A B7 FE 06 CC AD E5 A5 89 B1 01 00 04 56 93	.Z.....V.
0x000001F0	03 70 A5 85 02 24 93 8D 05 90 F0 0F 09 05 01	.p...\$......

○ PS C:\Users\QDEVP2\Desktop\Estructura Archivo Demo>

Esta imagen muestra el contenido interno de un archivo demo de Counter Strike, esta imagen no ha sido captura por mí mismo, es un simple ejemplo sobre la estructura interna del mismo, al ser archivos binarios para la visualización de su estructura se utilizan técnicas de visualización hexadecimal y técnicas de ASCII, muy similar a cuando se analiza un virus o un hackeo realizado por ciberdelincuentes.

COMO VISUALIZAR LOS ARCHIVOS DEMO

Los archivos demo solamente pueden ser representados para ver de nuevo la partida de manera gráfica como si de un video se tratara desde el propio motor del juego.

Es decir, si tú has descargado un archivo demo de una partida disputada con tu usuario y quieres volver a verla, la única manera de hacerlo es abriendo el archivo demo dentro del propio videojuego.

Pero si lo que queremos es realizar análisis de situaciones o de patrones repetitivos en cuanto a los videos de cámaras con inteligencia artificial podemos extraer dicha información y representarla mediante gráficas o tablas.

DONDE OBTENER LOS ARCHIVOS DEMO

La obtención de estos archivos demo del juego Counter Strike 2, se puede realizar de dos maneras bien diferenciadas, todo dependiendo del tipo de partida que queramos analizar.

ARCHIVOS DE PARTIDAS PROFESIONALES

Si nuestro objetivo es el análisis y la extracción de estadísticas de partidas profesionales, debemos acudir a la plataforma líder de aportación de archivo demo de todos y cada uno de los partidos profesionales que se disputan **HLTV**.

HLTV

HLTV es una página web cuyo objetivo es la muestra en tiempo real de los resultados de todos y cada uno de los partidos que se disputan a lo largo del día en el videojuego Counter Strike 2.



Si nuestro objetivo es la descarga de estos archivos demo hay que seguir el siguiente procedimiento:

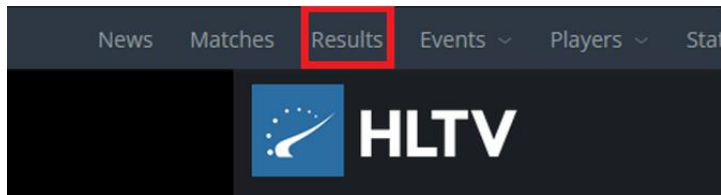
PASO 1

El primer paso será dirigirnos a la página oficial de HLTV:

<https://www.hltv.org/>

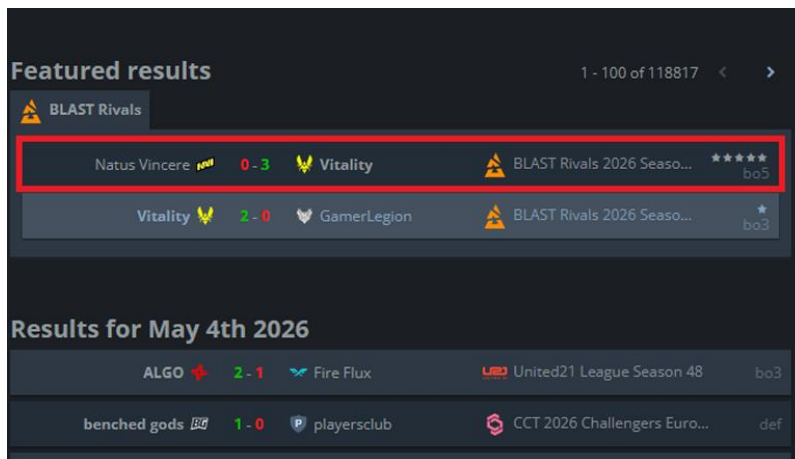
PASO 2

En la página oficial de HLTV debemos de irnos a la pestaña que aparece en la parte superior **“Results”**:



PASO 3

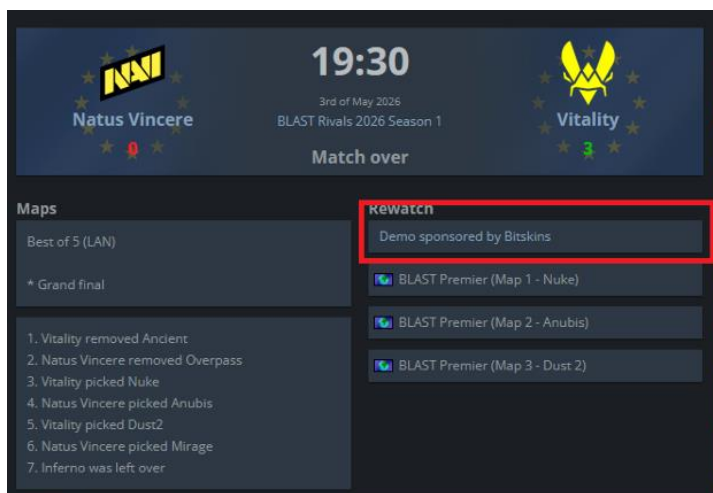
Una vez en se haya abierto la pestaña **“Results”**, elegiremos el partido del cual queremos obtener los archivos **.dem**.



PASO 4

La página de HLTV nos llevara a la página principal con las estadísticas del partido.

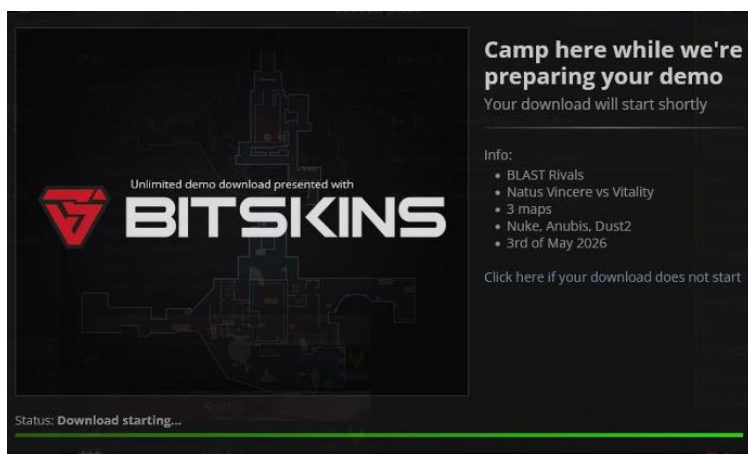
Una vez allí justo debajo del resultado a la derecha veremos los diferentes archivos de los diferentes mapas jugados en el encuentro.



PASO 5

Al hacer clic en este apartado comenzara una pantalla de carga con todas las indicaciones sobre el archivo comprimido que estamos descargando.

Este archivo comprimido contendrá un archivo **.dem** por cada uno de los mapas que se jugaron en el encuentro.



ARCHIVOS DE PARTIDAS JUGADAS POR EL USUARIO

Si nuestro objetivo es explotar un archivo de una partida en la cual hemos participado la metodología de descarga cambia.

Counter Strike 2 guarda estos archivos demo en un directorio en el ordenador en el cual tengamos instalado el videojuego y hayamos disputado la partida.

Para que esta forma de descarga sea posible debemos tener en cuenta que debemos tener el videojuego iniciado.

1. Haz clic en el ícono de "TV" (Ver partidas) en el menú de la izquierda.
2. Dirígete a la pestaña "Mis partidas".
3. Busca la partida que quieras descargar y haz clic en el botón "Descargar".
4. Una vez descargada, el archivo se guardará en tu ordenador y el botón cambiará a "Ver".
5. Los archivos descargados variaran su ruta de descarga dependiendo el sistema operativo y su versión, así como la configuración de dicho usuario en Steam.
6. Una ruta muy común es la siguiente:

[C:\Archivos de programa \(x86\)\Steam\steamapps\common\Counter-Strike Global Offensive\game\csgo\replays](C:\Archivos de programa (x86)\Steam\steamapps\common\Counter-Strike Global Offensive\game\csgo\replays)

Mas información sobre los archivos demo:

<https://www.online-convert.com/es/formato-de-archivo/dem>

<https://filext.com/es/extension-de-archivo/DEMO>

ANÁLISIS APLICACIÓN DEMOSCAN

La siguiente información elaborada para este documento tiene relación con la elaboración de una aplicación web elaborada con el **framework Laravel** con la función de **explotar y extraer** información de los **archivos demos** de Counter Strike 2 de los cuales se ha hablado anteriormente.

Esto es un estudio realizado previamente al desarrollo de la aplicación web para asegurar un desarrollo seguro y eficaz de la misma.



CÁTALOGO DE REQUISITOS

La presente aplicación web ha sido desarrollada utilizando el framework **Laravel** y tiene como objetivo permitir el análisis automático de archivos demo de partidas de Counter Strike 2 (CS2), mostrando estadísticas detalladas de los jugadores participantes.

Para el procesamiento de archivos demo se utiliza la librería:

<https://github.com/LaihoE/demoparser>

OBJETIVOS DE LA APLICACIÓN

Los principales objetivos de la aplicación son:

- Permitir el registro y autenticación de usuarios.
- Facilitar la subida de archivos demo de CS2.
- Procesar automáticamente los archivos demo.

- Mostrar estadísticas detalladas de la partida analizada.
- Centralizar toda la información en una interfaz web sencilla e intuitiva.

REQUISITOS FUNCIONALES

REQUISITO FUNCIONAL 1 – REGISTRO DE USUARIOS

El sistema debe permitir a nuevos usuarios registrarse mediante un formulario de creación de cuenta.

Entradas:

- Nombre de usuario
- Correo electrónico
- Contraseña

Salidas:

- Creación de cuenta en la base de datos.
- Confirmación de registro correcto.
- Inicio de sesión automático.

REQUISITO FUNCIONAL 2 – INICIO DE SESIÓN

El sistema debe permitir a los usuarios autenticarse mediante correo electrónico y contraseña.

Entradas:

- Nombre de usuario
- Contraseña

Salidas:

- Acceso al panel principal.
- Mensajes de error en caso de credenciales inválidas.

REQUISITO FUNCIONAL 3 – CIERRE DE SESIÓN

El sistema debe permitir al usuario cerrar sesión de forma segura.

Resultado esperado:

- Eliminación de la sesión activa.

- Redirección a la pantalla de inicio de sesión.

REQUISITO FUNCIONAL 4 – SUBIDA DE ARCHIVO DEMO

El sistema debe permitir a los usuarios subir archivos demo de Counter Strike 2.

Restricciones:

- Solo se aceptarán archivos compatibles con demos de CS2.
- El sistema validará la integridad del archivo antes del procesamiento.

Resultado esperado:

- Almacenamiento temporal del archivo.
- Inicio automático del proceso de análisis.

REQUISITO FUNCIONAL 5 – PROCESAMIENTO DE DEMOS

El sistema debe procesar los archivos demo utilizando la librería demoparser.

Funcionalidades:

- Lectura de eventos de la partida.
- Guardado de contenido Json en la base de datos.
- Extracción de estadísticas de jugadores.
- Obtención de datos relevantes de rendimiento.

Resultado esperado:

- Generación de estadísticas estructuradas.

REQUISITO FUNCIONAL 6 – VISUALIZACIÓN DE ESTADÍSTICAS DE PARTIDAS

Una vez procesado el archivo demo, el sistema debe mostrar una tabla con estadísticas de los jugadores participantes.

Información mostrada:

- Nombre del jugador
- Kills
- Deaths

- Assists
- ADR
- Headshots
- Rating
- Equipo
- Visualización de métricas avanzadas.

REQUISITOS NO FUNCIONALES

REQUISITO NO FUNCIONAL 1 – RENDIMIENTO

El sistema debe procesar archivos demo en un tiempo razonable dependiendo del tamaño del archivo.

REQUISITO NO FUNCIONAL 2 – SEGURIDAD

La aplicación debe proteger las rutas privadas mediante autenticación y control de sesiones.

REQUISITO NO FUNCIONAL 3 – ESCALABILIDAD

La arquitectura debe permitir futuras ampliaciones funcionales, como:

- Nuevos tipos de estadísticas.
- Comparativas entre jugadores.
- Rankings automáticos.

REQUISITO NO FUNCIONAL 4 – USABILIDAD

La interfaz debe ser intuitiva y accesible para usuarios con conocimientos básicos de informática.

REQUISITO NO FUNCIONAL 5 – COMPATIBILIDAD

La aplicación debe funcionar correctamente en navegadores modernos:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

REQUISITO NO FUNCIONAL 6 – MANTENIBILIDAD

El código debe seguir una estructura modular basada en el patrón MVC proporcionado por Laravel.

SEGUIMIENTO DEL DESARROLLO

El seguimiento del desarrollo de la aplicación web **DemoScan** se puede llevar a cabo a través del repositorio de git público el cual se proporciona el enlace a continuación:

<https://github.com/alejandrohurga/AHFDWESProyectoFinalLaravel>

El repositorio cuenta con dos ramas (master y developerAHF), las cuales cada una de ellas tiene una funcionalidad dentro del control de versiones:

- **master:** Rama la cual solamente almacena los merges de la rama developerAHF, previos a la creación de una reléase para subir a explotación.
- **developerAHF:** Rama en la cual se realiza todo el desarrollo y evolución de la aplicación, así como las pruebas pertinentes antes de hacer el merge con la rama master.

ESTRUCTURA DE ESTUDIO

Para asegurar un desarrollo seguro y eficaz de la aplicación se ha realizado un estudio previo el cual tiene una estructura clara y fragmentada, cubriendo así los diferentes aspectos de un desarrollo seguro.

El estudio contiene estos dos pilares centrales los cuales son esenciales y obligatorios:

1. Diagrama de casos de uso.
2. Modelo Físico de Datos.

Cada apartado tiene un estudio y explicación detallada para el completo entendimiento de su elaboración.

DIAGRAMA DE CASOS DE USO

A continuación, se muestra toda la información relevante en cuanto al diagrama de casos de uso hasta el momento de la aplicación desarrollada para el proyecto final del ciclo de grado superior de desarrollo de aplicaciones web.

OBJETIVOS Y METAS

Los objetivos de la siguiente información son los siguientes:

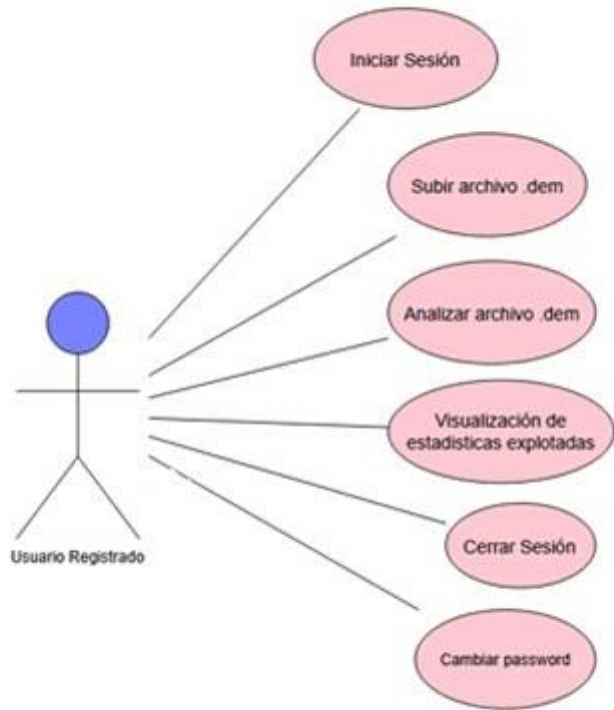
- Mejorar la elaboración del proyecto.
- Aumentar la seguridad de la aplicación.
- Dejar claras las acciones de cada tipo de usuario.

CASOS DE USO DE UN USUARIO REGISTRADO

La siguiente tabla indica las acciones que puede realizar un usuario registrado en la aplicación web:

Acciones	Descripción
Iniciar Sesión	<i>Un usuario debe de tener previamente una cuenta antes de Iniciar Sesión.</i>
Subir archivo demo	<i>El usuario puede adjuntar un archivo demo al selector de archivos para su explotación.</i>
Analizar el archivo demo	<i>Una vez subido el archivo el usuario puede analizar y realizar la explotación de dicho archivo.</i>
Visualización de estadísticas exportadas	<i>Tras la explotación de dicho archivo el usuario puede visualizar los resultados de la explotación.</i>
Cerrar Sesión	<i>Un usuario registrado que a iniciado sesión puede cerrar sesión en su propia cuenta.</i>
Cambiar <u>Password</u>	<i>Un usuario registrado que ha iniciado sesión puede cambiar su contraseña desde el perfil de usuario.</i>

REPRESENTACIÓN GRÁFICA DIAGRAMA DE CASOS DE USO USUARIO REGISTRADO

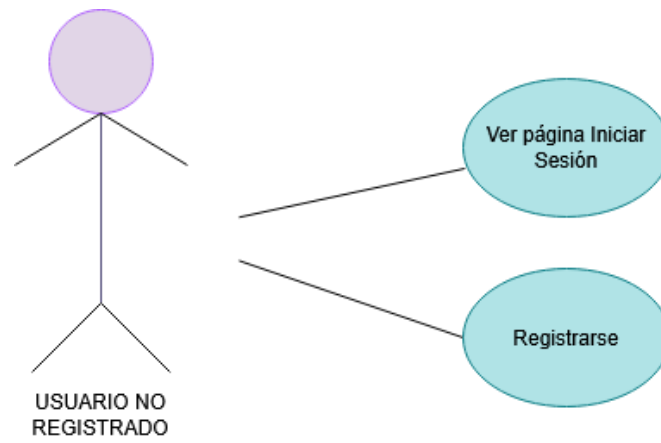


CASO DE USO DE USUARIOS NO REGISTRADOS

La siguiente tabla indica las acciones que puede realizar un usuario no registrado en la aplicación web:

Acciones	Descripción
Visualizar página de iniciar sesión.	<i>Si el usuario no esta registrado la única página que podrá visualizar será el formulario de Iniciar Sesión.</i>
Registrarse.	<i>Un usuario no registrado tendrá la opción de acceder al formulario de registro mediante un botón en la parte inferior del formulario de iniciar sesión.</i>

REPRESENTACIÓN GRÁFICA DIAGRAMA DE CASOS DE USO USUARIO NO REGISTRADO



MODELO FÍSICO DE DATOS

La siguiente información incluye la elaboración del modelo físico de la base de datos utilizada para la persistencia y administración de los datos utilizados en la aplicación web, desarrollada como proyecto final del ciclo de grado superior de desarrollo de aplicaciones web.

La aplicación consta de dos tablas relacionadas entre sí las cuales representan usuarios y análisis, su relación es 1:N debido a que un usuario puede tener varios análisis realizados y una tabla autónoma la cual es alimentada mediante el "scrapeo" de otra página web la cual nos permite mostrar los jugadores y sus estadísticas.

Antes de visualizar el diagrama vamos a realizar una explicación detallada sobre cada una de las 3 tablas que forman el diagrama. El siguiente diagrama ha sido realizado mediante un estudio previo de la aplicación desarrollada para implementar una estructura de datos ordenada y optimizada para las diferentes funciones de la aplicación.

TABLAS DE LA BASE DE DATOS

TABLA USUARIOS

La tabla usuarios representa a todos los usuarios que tienen cuenta en la aplicación, permitiendo guardar en ella toda la información relevante en cuanto a cada usuario como su id (Clave primaria), nombre de usuario o contraseña cifrada lo cual nos permite hacer un inicio de sesión seguro. Esta tabla tiene relación 1:N con la tabla "analises" debido a que la subida de cada uno de los demos para su análisis está vinculado a una cuenta de usuario.

Usuarios
<u>ID INT AUTOINCREMENT</u>
Nombre: String Correo: String Password: String fecha_verificacion_correo: DATETIME created_at: DATETIME updated_at: DATETIME

- **ID:** Representa la primary key del usuario.
- **Correo:** Correo electrónico de la cuenta del usuario.
- **Password:** Contraseña de la cuenta del usuario.
- **Fecha_verificacion_correo:** Fecha en la que se verifico el correo electrónico de la cuenta.
- **Created_at:** Fecha de creación de la cuenta de usuario. (Automático de Laravel).
- **Updated_at:** Fecha de actualización de la cuenta de usuario , por defecto la misma que la de creación.

TABLA ANALISES

La tabla “**analises**” (análisis), guarda toda la información relacionada con los análisis realizados tras la subida de los archivos demos a la plataforma.

Esta tabla no almacena el archivo demo sin analizar ni procesar, tras el estudio realizado se llegó a la conclusión de que almacenar este tipo de archivos tan grandes no era óptimo para gestionar la base de datos. Como solución esta tabla almacena el análisis de dicho archivo en formato JSON (Columna Stats), lo cual permite una mayor fluidez y optimización de la aplicación a la hora de transportar y mostrar los datos.

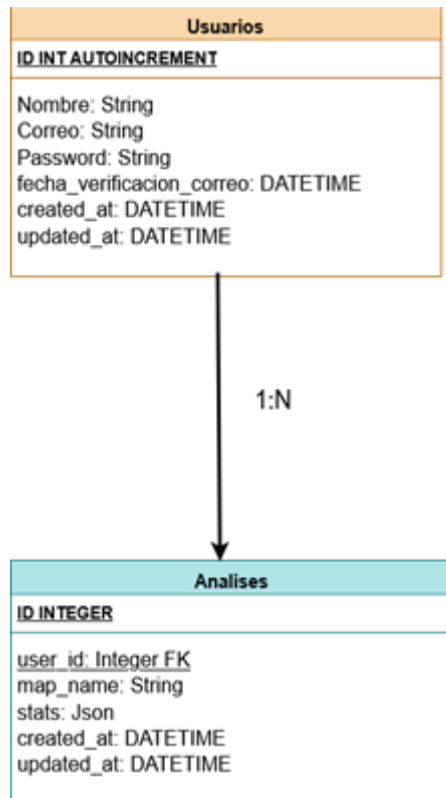
Esta tabla tiene una relación **1:N** con la tabla usuarios debido a que un análisis solamente puede estar vinculado a un único usuario.

Analises
ID INTEGER
<u>user_id</u> : Integer FK map_name: String stats: Json created_at: DATETIME updated_at: DATETIME

- **ID:** Es la primary key de cada análisis subido por los usuarios.
- **user_id:** Es la clave foránea, la cual representa el id del usuario, ya que no existen análisis que no estén subidos por ningún usuario.
- **map_name:** Nombre del mapa en el cual se ha jugado la partida.
- **Stats:** Campo Json el cual guarda las estadísticas de los jugadores en la partida para luego mostrarlas en la vista.

- **Created_at:** Fecha de creación de la cuenta de usuario. (Automático de Laravel).
- **Updated_at:** Fecha de actualización de la cuenta de usuario, por defecto la misma que la de creación.

MODELO DE DATOS COMPLETO



TECNOLOGÍAS UTILIZADAS

A continuación, tenemos una tabla la cual muestra todas las tecnologías utilizadas para el desarrollo de dicha aplicación web:

Capa	Tecnología	Papel
Backend	PHP 8.2+ / Laravel 12	Lógica de aplicaciones, autenticación y enrutamiento.
Motor de análisis	Node.js / demoparser2	Procesamiento de demostración binaria de alta velocidad.
Frontend	Hoja / Tailwind CSS / Alpine.js	IU reactiva y estilo.
Herramienta de construcción	Vite	Agrupación de activos y HMR.
Base de datos	MySQL	Persistencia para usuarios, demostraciones y resultados.

- **PHP 8.2+ / Laravel 12:** La funcionalidad de Laravel 12 en nuestra aplicación final será la de procesar toda la lógica de negocio, así como la seguridad de manera automática gracias a la implementación de Breeze.
- **Node JS:** Es un entorno de ejecución de JavaScript de código abierto y multiplataforma, basado en el motor V8 de Chrome, que permite ejecutar código JS fuera del navegador web, principalmente en servidores, la funcionalidad de node en nuestra aplicación es la de explotar y extraer la información de los archivos demos subidos mediante la librería demoparser2.
- **Tailwind:** La aplicación utiliza Tailwind para el desarrollo de los estilos y el frontend ya que se implementa con facilidad dentro de los archivos Blade de Laravel.
- **MySQL:** Se ha optado por la base de datos por defecto de Laravel para la gestión y persistencia de los datos.

NODE JS Y LIBRERÍA DEMOPARSER

El corazón de la aplicación web que estamos diseñando este compuesto de dos elementos muy bien diferenciados:

NODE JS

- **Qué es:** Node JS es un entorno de ejecución el cual permite la ejecución de código JavaScript fuera del navegador web, lo que permite que el desarrollo con JavaScript vaya mas allá de la programación del lado del cliente.
- **Funcionalidad en la aplicación:** El papel de Node JS en nuestra aplicación web es de suma importancia debido a que es el encargado de poder ejecutar el script de análisis de los archivos demo en el lado del servidor. Esto nos permite el análisis y el guardado de los datos extraídos en nuestra base de datos.

LIBRERÍA DEMOPARSER

La librería demoparser es otra parte fundamental de la aplicación web ya que gracias a ella podemos dotar a los scripts de extracción de datos de métodos y funciones optimizados para la extracción de los datos.

Fuente de la librería: Esta librería y toda su documentación puedes encontrarla en el siguiente repositorio de GitHub.

URL: <https://github.com/LaihoE/demoparser.git>

Autor: LaihoE

El script de extracción de datos en nuestra aplicación se encuentra en el siguiente directorio:

Storage/scripts/demoparser/parse.cjs

ARQUITECTURA GENERAL DEL SISTEMA

La aplicación sigue una arquitectura cliente-servidor basada en MVC (Modelo-Vista-Controlador).

Flujo principal:

1. El usuario accede al sistema.
2. El usuario se autentica.
3. El usuario sube un archivo demo.

4. Laravel recibe el archivo.
5. El backend procesa el demo mediante demoparser.
6. Se generan estadísticas estructuradas.
7. Los resultados se muestran en la interfaz web.

FLUJO DEL ARCHIVO DEMO EN LA APLICACIÓN

A continuación, vamos a ver el flujo del archivo demo desde que es subido por el usuario a la aplicación web hasta el guardado de la información en la tabla de la base de datos.

1. El archivo es subido a la aplicación web mediante un formulario de un solo campo (Selector de archivos).
2. Una vez subido el archivo este es almacenado en el siguiente directorio: **storage/app/private/demos**
3. El archivo se mantendrá aquí mientras es analizado, aquí entra en acción el script **"parse.cjs"**, el cual es el encargado de analizar este script subido de nuevo, ya que ya ha sido analizado en el momento de subirlo, y exportar la información a JSON, utiliza el método **process.stdout** para poder administrar el archivo desde PHP.
4. Una vez analizado y mostrados los resultados es momento del controlador **AnalisisController** que será el encargado de guardar ese archivo JSON en una columna de la base de datos.
5. Una vez analizado y guardada la información en nuestra base de datos, de nuevo el controlador **AnalisisController** será el **encargado de borrar el archivo demo**, hasta ahora almacenado en el directorio storage.
6. Ejemplo de estructura del JSON:

```
1  [
2
3      {
4          "mvps": 2,
5          "name": "ADRON",
6          "tick": 261283,
7          "score": 38,
8          "steamid": "76561198258044780",
9          "kills_total": 18,
10         "deaths_total": 23,
11         "ace_rounds_total": 0,
12         "headshot_kills_total": 10
13     },
14     {
15         "mvps": 3,
16         "name": "alex",
17         "tick": 261283,
18         "score": 50,
19         "steamid": "76561198000984547",
20         "kills_total": 21,
21         "deaths_total": 20,
22         "ace_rounds_total": 0,
23         "headshot_kills_total": 9
24     },
25     {
26         "mvps": 4,
27         "name": "ragga",
28         "tick": 261283,
```

Esta es la estructura del archivo JSON una vez analizado el archivo y que tiene los siguientes campos:

- **Mvps:** Número de veces que ha sido el jugador el mejor de la ronda.
- **Name:** Nombre del jugador dentro del videojuego.
- **Tick:** Tick en el cual se ha hecho la última lectura.
- **Score:** Puntuación del jugador en la partida.
- **Steamid:** Id de la cuenta del juego.
- **Kills_total:** Bajas totales del jugador.
- **Deaths_total:** Muertes totales del jugador.
- **Ace_rounds_total:** Número de veces que el jugador mato a todos los rivales en una ronda.
- **Headshot_kills_total:** Numero de veces que el jugador mato a un rival de un tiro en la cabeza.

Toda esta información se almacena en la columna **stats** de la tabla **analises** en la base de datos de la aplicación.

La realización del almacenamiento de esta manera es debido a que como primera versión de la aplicación he decidido centrar el desarrollo y estudio al principio en el extractor de datos de manera correcta.

DISEÑO Y DESARROLLO

PREPARACIÓN ENTORNO DE DESARROLLO

En cuanto al entorno de desarrollo vamos a llevar una guía a través de los requisitos necesarios para el desarrollo de una aplicación Laravel como la que hemos desarrollado para nuestro proyecto.

El entorno de desarrollo girará a partir de una pieza centra que será Laragon, según como vimos en la tabla de tecnologías utilizadas de nuestra aplicación

Como podemos ver para desarrollar la aplicación hemos necesitado Php 8.3, Laravel 12 y Composer.

Composer y Php se vienen instalados en **Laragon** de manera automática además de los servicios como Apache y MySQL los cuales se inician automáticamente.

A parte nuestra aplicación necesitara Node JS y npm para la explotación de los archivos demos.

ENTORNO DESARROLLO LOCAL (LARAGON)

Para comenzar con el desarrollo de nuestra **Aplicación Web con Laravel** a medida que realizamos un estudio del propio Laravel, necesitamos un entorno de desarrollo de manera local que nos permita el desarrollo de manera dinámica.

En cuanto a mi elección he decidido realizar el desarrollo de manera local mediante el entorno de desarrollo local **Laragon**.



LARAGON

Laragon es un entorno de desarrollo local que incluye:

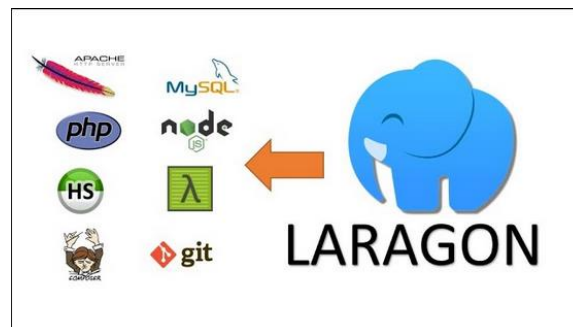
- Apache
- PHP
- MySQL
- Herramientas para proyectos Laravel

Además, permite crear entornos locales de manera rápida y sencilla.

Laragon nos permite el cambio de versiones de manera dinámica y rápida lo cual hace que al trabajar con múltiples librerías o dependencias sea muy fácil el desarrollo de nuestra aplicación.

¿POR QUÉ LARAGÓN?

La elección de utilizar Laragon de manera local **y no una máquina virtual** la cual tuviera un **servidor Apache** es por sencillez y comodidad.



Pero se **resume en los siguientes aspectos** los cuales **se han tenido en cuenta** a la hora de planificar el montaje y administración del entorno de desarrollo:

1. **Eliminación de la gestión de permisos** al crear una aplicación Laravel.
2. **Sistema unificado** en cuanto a la hora de desarrollar en la empresa.
3. Control y **cambio de versiones de manera instantánea**.
4. **Instalación** de aplicación Laravel de manera **rápida e intuitiva**.
5. Incluye e incorpora un servidor **Apache/Nginx**.
6. Facilita el cambio entre versiones de **PHP y MySQL**.
7. Incorpora herramientas como la consola (**Cmd**) o **HeidiSQL** para gestionar la base de datos.

Debido a todos estos aspectos se ha elegido Laragon como entorno de desarrollo de esta Aplicación Final.

FUNCIONAMIENTO DE LARAGON

En este apartado vamos a ver como descargar, instalar y ejecutar Laragon de manera eficaz conociendo todas sus funciones.

1. Descarga de Laragon:

Para realizar la descargar debemos de hacerlo en la página oficial de Laragon, podéis encontrar el enlace a continuación:

<https://laragon.org/download>

2. Instalación Laragon

En cuanto a la instalación lo único que debemos hacer es avanzar en el ejecutable .exe que acabamos de descargar hasta que empiece la instalación de la aplicación.

3. Ejecutar Laragon

Laragon se ejecutará siempre en segundo plano en condiciones normales para poder acceder a el simplemente debemos buscarlo en el menú inferior de nuestro ordenador.

4. Creación de aplicación Laravel con Laragón:

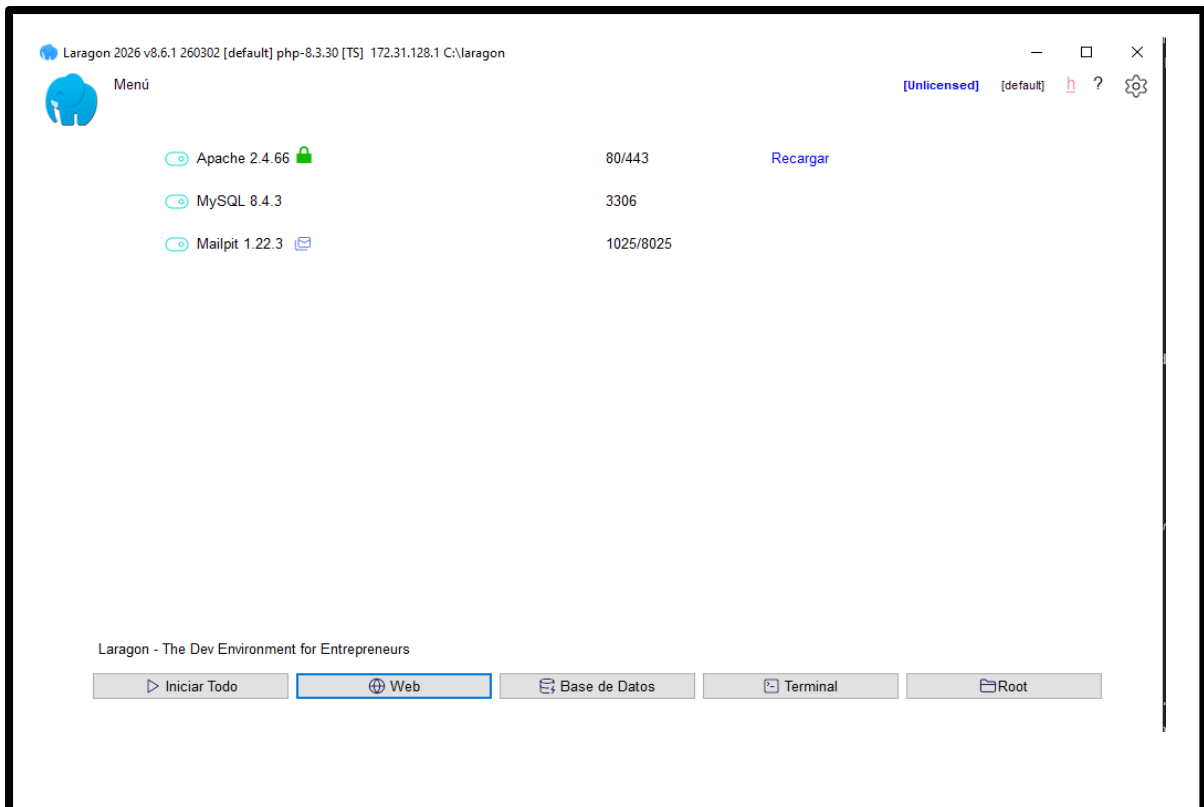
*Para realizar la creación de una aplicación Laravel mediante el entorno de desarrollo local Laragon, lo único que debemos hacer será dirigirnos al apartado que se encuentra arriba a la derecha el cual dice **"Menu"**, una vez desplegado el submenú haremos clic en la opción que dice **"Creación rápida de sitios web"**, esto nos abrirá un submenú en el cual elegiremos que tipo de aplicación queremos crear (En nuestro caso **Laravel**).*

Esto nos creara un proyecto Laravel con el nombre especificado en el siguiente directorio que trae por defecto configurado Laragon:

Este Equipo/Disco Local (C:)/Laragon/www/

5. Interfaz Laragon

Laragon presume de tener una interfaz sencilla e intuitiva en todo momento en la cual se diferencia perfectamente la funcionalidad de cada uno de los botones:



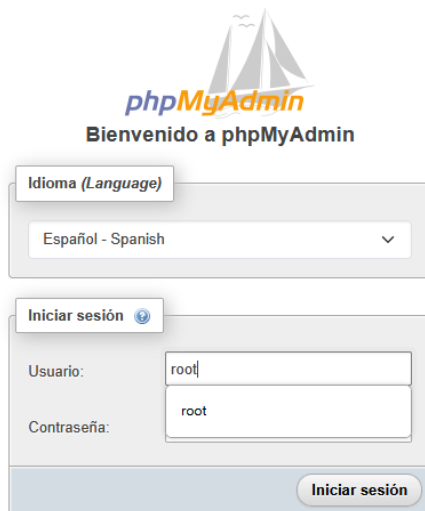
ACCEDER AL SISTEMA GESTOR DE BASES DE DATOS (PHPMYADMIN)

Laragon utiliza como gestor de base de datos **PhpMyAdmin** para acceder a este sistema únicamente debemos de dirigirnos al botón que dice "Base de Datos", en la parte inferior de la pantalla.



Una vez hacemos clic en el botón nos abre una nueva pestaña en el navegador en la página web de **phpMyAdmin** (Inicio de sesión).

Como estamos en desarrollo local las credenciales para acceder a la gestión de la base de datos son "**root**" y el campo de la contraseña totalmente vacío.



The image shows the phpMyAdmin login interface. At the top, there is a logo with a sailboat and the text "phpMyAdmin". Below the logo, it says "Bienvenido a phpMyAdmin". There is a language selection dropdown menu set to "Español - Spanish". Below that is a login form with fields for "Usuario:" (containing "root") and "Contraseña:" (containing "root"). A "Iniciar sesión" button is at the bottom right of the form.

Una vez hemos iniciado sesión se nos presentara la interfaz de phpMyAdmin donde podemos interactuar y gestionar todas nuestras bases de datos.



DESARROLLO DE LA APLICACIÓN

Toda la información aportada a continuación tiene que ver con el desarrollo de la aplicación web DemoScan.

Para ello se utilizarán ejemplos de código, así como la detallada explicación sobre cada función desarrollada para la aplicación.

Todo el desarrollo de la aplicación se puede seguir desde el siguiente repositorio:

<https://github.com/alejandrohurga/AHFDWESProyectoFinalLaravel>

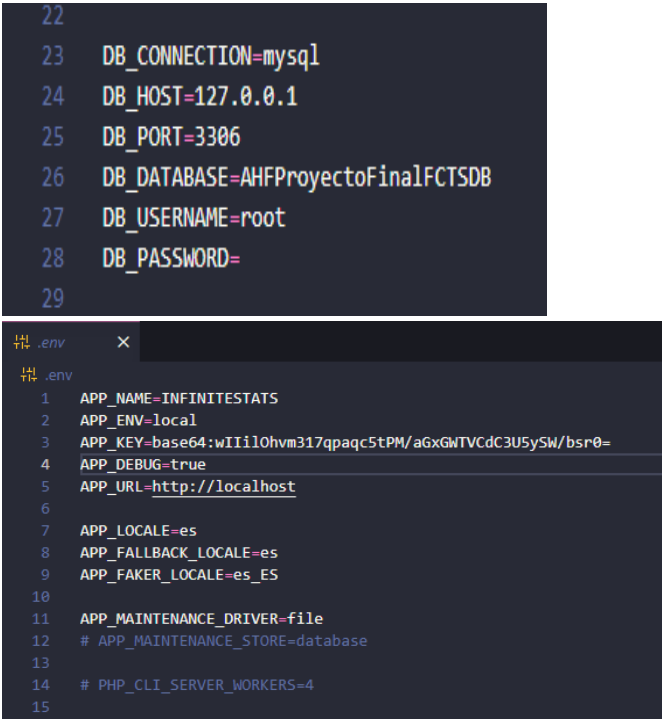
CREACIÓN DEL PROYECTO

La creación del proyecto la hemos realizado mediante el siguiente comando:

`composer create-project laravel/laravel cs2-demo-parser`

CONFIGURACIÓN DEL ARCHIVO .ENV

Configuramos el archivo .env para poder realizar la conexión con la base de datos, así como la zona horaria de la aplicación:



```
22
23 DB_CONNECTION=mysql
24 DB_HOST=127.0.0.1
25 DB_PORT=3306
26 DB_DATABASE=AHFProyectoFinalFCTSDb
27 DB_USERNAME=root
28 DB_PASSWORD=
29

1 APP_NAME=INFINITESTATS
2 APP_ENV=local
3 APP_KEY=base64:wIi10hvm317qpaqc5tPM/aGxGWTVCdC3U5ySW/bsr0=
4 APP_DEBUG=true
5 APP_URL=http://localhost
6
7 APP_LOCALE=es
8 APP_FALLBACK_LOCALE=es
9 APP_FAKER_LOCALE=es_ES
10
11 APP_MAINTENANCE_DRIVER=file
12 # APP_MAINTENANCE_STORE=database
13
14 # PHP_CLI_SERVER_WORKERS=4
15
```

Los datos de este archivo debemos de cambiarlos con la información adecuada de nuestro servidor de explotación cuando subamos nuestra aplicación.

GENERACIÓN DE LA CLAVE LARAVEL

Laravel requiere una clave de cifrado para el funcionamiento de sesiones y seguridad.

La cual generamos mediante el siguiente comando:

"php artisan key:generate"

INSTALACIÓN Y DESARROLLO DE SISTEMA DE AUTENTICACIÓN

Para poder incorporar el sistema de autenticación que tiene por defecto Laravel (Breeze) lo hacemos ejecutando el siguiente comando:

"php artisan breeze:install"

Hemos editado y modificado el sistema de inicio de sesión que implanta Breeze por defecto, en el cual iniciamos sesión mediante correo electrónico y contraseña.

La actualización cambia dicho inicio de sesión por uno mucho mas simple mediante el cual se ingresa a la aplicación mediante los datos de nombre de usuario y contraseña.

Para ello hemos desarrollado los siguientes archivos:

- **Creación de la migración usuarios:**

```
public function up(): void
{
    // Crea la tabla de la Base de Datos de usuarios.
    Schema::create('usuarios', function (Blueprint $table) {
        // Poniendo id, Laravel nos creara una columna BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY
        $table->id();
        $table->string('nombre')->unique();
        $table->string('correo');
        $table->string('password');
        $table->timestamp('fecha_verificacion_correo')->nullable(); // Así no falla si no se rellena.
        $table->rememberToken(); // Para realizar la función de "Recuerdame"
        $table->timestamps(); // Esto crea created_at y updated_at automáticamente.
    });

    // Crea la tabla de la base de datos password_reset_tokens.
    Schema::create('password_reset_tokens', function (Blueprint $table) {
        $table->string('email')->primary();
        $table->string('token');
        $table->timestamp('created_at')->nullable();
    });

    // Crea la tabla de la base de datos de sessions
    Schema::create('sessions', function (Blueprint $table) {
        $table->string('id')->primary();
        $table->foreignId('user_id')->nullable()->index();
        $table->string('ip_address', 45)->nullable();
        $table->text('user_agent')->nullable();
        $table->longText('payload');
        $table->integer('last_activity')->index();
    });
}
```

Esta tabla será la nueva tabla de usuarios, por defecto Breeze usa users.

- **Creación del modelo Usuario:**

```
class Usuario extends Authenticatable
{
    use Notifiable;

    // Nombre de la tabla en la base de datos
    // Lo ponemos explícito porque Laravel por defecto buscaría 'usuarios'
    protected $table = 'usuarios';

    protected $fillable = [
        'nombre',
        'correo',
        'password',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    protected function casts(): array
    {
        return [
            'fecha_verificacion_correo' => 'datetime',
            'password' => 'hashed',
        ];
    }
}
```

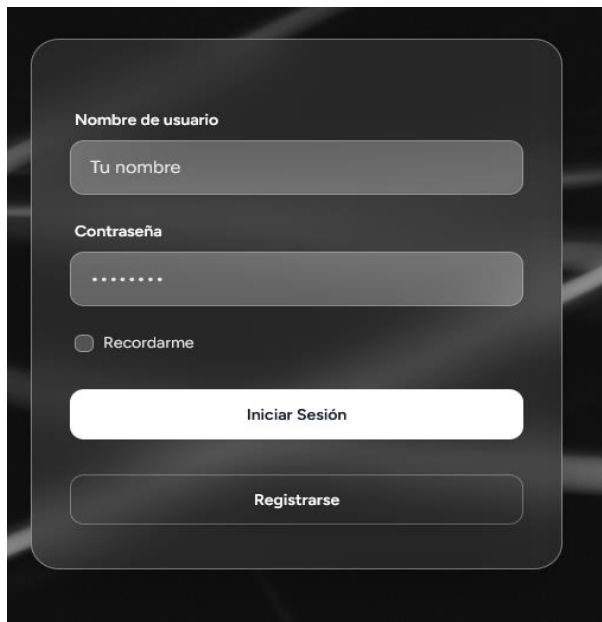
- **Controlador Registro**

Actualizamos el controlador de registro para que valide por nombre de usuario el cual lo encontramos en la siguiente ruta:

App/http/controllers/auth/registeredUserController.php

```
$request->validate([
    'nombre' => ['required', 'string', 'max:255', 'unique:usuarios'],
    'correo' => ['required', 'string', 'max:255'],
    'password' => ['required', 'confirmed', Rules\Password::defaults()],
]);
```

El inicio de sesión ahora nos queda de la siguiente manera:



The image shows a login form on a dark background. It contains two input fields: 'Nombre de usuario' with the placeholder 'Tu nombre' and 'Contraseña' with masked characters. Below these is a checkbox labeled 'Recordarme'. At the bottom are two buttons: 'Iniciar Sesión' (white with black text) and 'Registrarse' (dark with white text).

INSTALACIÓN DE DEPENDENCIAS DEL CLIENTE

Lo siguiente que realizamos será la instalación de todas las dependencias necesarias del lado del cliente mediante los siguientes comandos:

npm install

npm run dev

DESARROLLO SCRIPT DE ANÁLISIS

Para el desarrollo del script de javascript el cual es el encargado de la explotación de los archivos demos se han utilizado los diversos métodos de los cuales nos dota la librería demoparser:

Para ello la librería se instaló en el siguiente directorio dentro de la estructura de nuestro proyecto Laravel:

Storage/scripts/demoparser

El archivo de análisis del archivo se encuentra en el mismo directorio con el nombre **parse.cjs**.

Contenido del archivo:

```
1  const { parseEvent, parseTicks } = require('@laihoe/demoparser2');
2
3  const pathToDemo = process.argv[2];
4
5  if (!pathToDemo) {
6    process.exit(1);
7  }
8
9  try {
10   const roundEnds = parseEvent(pathToDemo, "round_end");
11
12   // Si la demo está vacía o corrupta, evitamos el error de Math.max
13   if (!roundEnds || roundEnds.length === 0) {
14     process.exit(1);
15   }
16
17   const gameEndTick = Math.max(...roundEnds.map(x => x.tick));
18
19   // Añadimos 'user_name' para que la tabla no salga vacía
20   const fields = [
21     "name",
22     "kills_total",
23     "deaths_total",
24     "mvps",
25     "headshot_kills_total",
26     "ace_rounds_total",
27     "score"
28   ];
29
30   const scoreboard = parseTicks(pathToDemo, fields, [gameEndTick]);
31
32   // IMPORTANTE: Imprimimos SOLO el JSON.
33   process.stdout.write(JSON.stringify(scoreboard));
34
35   } catch (e) {
36     // Los errores van al canal de error, no al de salida (stdout)
37     process.stderr.write(e.message);
38     process.exit(1);
39   }
40 }
```

DESARROLLO DEL LAS RUTAS

Para el desarrollo de la aplicación se han codificado una serie de rutas en el archivo web.php el cual se encuentra en el siguiente directorio:
/routes

Contenido del archivo:

```
27
28 Route::get('/dashboard', function () {
29     return view('dashboard');
30 })->middleware(['auth', 'verified'])->name('dashboard');
31
```

Ruta la cual carga la vista dashboard.blade.php

```
57
58 Route::post('/demo/guardar', [DemoController::class, 'guardarArchivo'])->name('demo.guardar')->middleware(['auth', 'verified']);
59
60 Route::middleware('auth')->group(function () {
61     Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
62     Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
63     Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
64 });
65
66
67 require __DIR__.'/auth.php';
68
```

Ruta la cual administra el guardado del archivo demo, llama al método del controlador (DemoController.php) guardarArchivo.

Es una ruta solamente disponible para usuarios autenticados por eso utilizamos el require, para evitar el acceso de usuarios no logueados.

DESARROLLO DE LOS CONTROLADORES

Para el desarrollo de la lógica de la aplicación se han codificado algunos métodos para poder realizar el análisis de estos archivos demos que subimos a la aplicación.

Ejemplo de código del método store() del AnalisisController:

```
// 1. Validar la subida del archivo
$request->validate([
    'demo_file' => 'required|file|max:100000', // ~100MB
]);
```


Mediante el método **validate()** comprobamos que el archivo no pese más que el tamaño especificado.

```
try {
    // 2. Guardar el archivo temporalmente
    // Usamos el disco 'local' (storage/app)
    $path = $request->file('demo_file')->store('temp_demos');
    $fullPath = storage_path('app/' . $path);

    // 3. Ejecutar el script de Node.js
    $scriptPath = base_path('scripts/parse_demo.js');

    // Usamos Process de Laravel para capturar la salida
    $result = Process::run("node \"$scriptPath\" \"$fullPath\"");

    if (!$result->successful()) {
        return back()->with('error', 'Error en el parser: ' . $result->errorOutput());
    }

    // 4. Decodificar el JSON que escupe el script de Node
    $datosJson = json_decode($result->output(), true);

    if (empty($datosJson)) {
        return back()->with('error', 'El parser devolvió un JSON vacío.');
```

Mediante este bloque de código el archivo de guarda temporalmente para su análisis, se extrae la información en formato Json.

```
// 5. Guardar en la base de datos
$nuevoAnálisis = new Analisis();
$nuevoAnálisis->user_id = Auth::id();
$nuevoAnálisis->map_name = "Inferno"; // Luego puedes dinamizar esto si el JSON trae el mapa
$nuevoAnálisis->stats = $datosJson; // IMPORTANTE: El modelo debe tener: protected $casts = ['stats' => 'array'];
$nuevoAnálisis->save();

// 6. Limpiar: Borrar el archivo .dem temporal para no llenar el disco
if (file_exists($fullPath)) {
    unlink($fullPath);
}
```

Guardamos el archivo en la base de datos y limpiamos el archivo subido.

```

class Analisis extends Model
{
    /**
     * Utilizamos el casting de Eloquent.
     * Nos permitira convertir el archivo JSON en una cadena de texto cada vez que usemos
     * una tupla de base de datos y asi poder mostrarla comodamente en la vista.
     */

    protected $casts = [
        'stats' => 'array',
    ];
}

```

En el modelo Analisis utilizamos el casting de Eloquent para poder transformar ese archivo JSON en una cadena de texto y así poder almacenar la información en la base de datos en la tabla analyses en la columna stats.

				id	user_id	map_name	stats	created_at	updated_at
☐	Editar	Copiar	Borrar	28	5	Nuke	[{"mvps": 2, "name": "ADRON", "tick": 261283, "sco...	2026-05-06 14:07:21	2026-05-06 14:07:21
☐	Editar	Copiar	Borrar	29	5	Nuke	[{"mvps": 2, "name": "ADRON", "tick": 261283, "sco...	2026-05-11 10:57:01	2026-05-11 10:57:01
☐	Editar	Copiar	Borrar	30	5	Anubis	[{"mvps": 3, "name": "sausol", "tick": 184033, "sc...	2026-05-11 15:26:43	2026-05-11 15:26:43
☐	Editar	Copiar	Borrar	31	5	Nuke	[{"mvps": 2, "name": "ADRON", "tick": 261283, "sco...	2026-05-13 10:29:30	2026-05-13 10:29:30
☐	Editar	Copiar	Borrar	32	5	Ancient	[{"mvps": 1, "name": "ADRON", "tick": 179049, "sco...	2026-05-13 10:32:50	2026-05-13 10:32:50
☐	Editar	Copiar	Borrar	33	5	Nuke	[{"mvps": 2, "name": "ADRON", "tick": 261283, "sco...	2026-05-18 09:12:22	2026-05-18 09:12:22
☐	Editar	Copiar	Borrar	34	5	Ancient	[{"mvps": 1, "name": "ADRON", "tick": 179049, "sco...	2026-05-18 09:13:15	2026-05-18 09:13:15

IMPLANTACIÓN DE LA APLICACIÓN

Una vez hecho el análisis para su posterior desarrollo vamos a finalizar este documento con la guía paso a paso para implantar nuestra aplicación en un servidor de explotación y así estar accesible para todo el mundo desde cualquier parte.

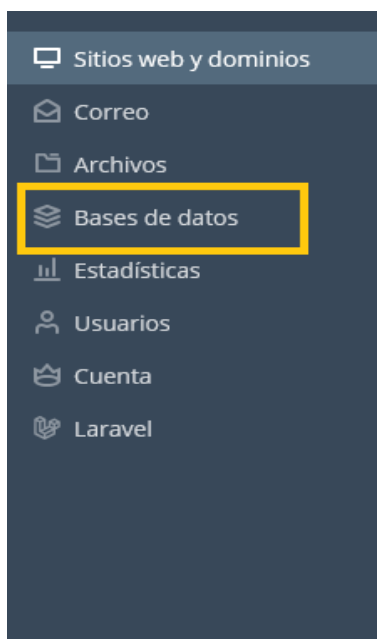
Dirección en explotación del proyecto:

AHFDWESDemoScan.alejandrohuefer.ieslossauces.es

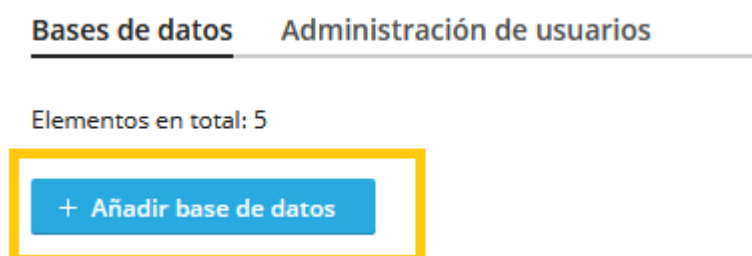
En la URL proporcionada se puede acceder a la aplicación web en el servidor de explotación.

IMPLANTACIÓN BASE DE DATOS

Lo primero que debemos es dirigirnos al apartado "Bases de datos" en el menú lateral izquierdo de Plesk.



Una vez aquí haremos click en el botón azul que dice "Agregar Base de datos".



Esto nos abrirá una sección lateral donde debemos de configurar los datos de la nueva base de datos:

Añadir base de datos

Nombre de la base de datos *

alejandrohuefer_

Servidor de bases de datos

localhost:3306 (predeterminado para MariaDB, v10.11.14)

Sitio relacionado

Ningún sitio relacionado

Usuarios

Cree un usuario predeterminado para la base de datos. Plesk accederá a la base de datos en nombre de este usuario. Si no se asigna ningún usuario de base de datos a la base de datos, no podrá accederse a la misma.

☒ Crear un usuario de la base de datos

Nombre de usuario de la base de datos *

Contraseña *

Generar

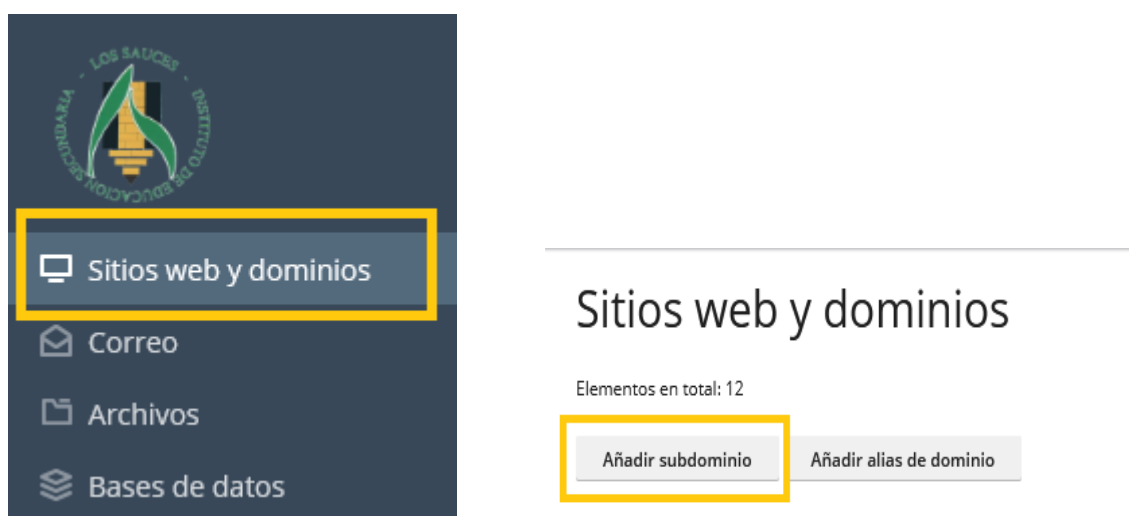
☐ El usuario tiene acceso a todas las bases de datos de la suscripción seleccionada

Debemos de recordar estos datos ya que los necesitaremos para realizar la configuración del archivo **.env** que generaremos más adelante.

CREACIÓN SUBDOMINIO

Una vez creada la base de datos lo que deberemos hacer será crear un subdominio para nuestra aplicación, para ello nos dirigimos de nuevo al menú lateral, esta vez elegiremos la opción que dice Sitios Web y dominios.

Una vez en la página debemos hacer click en **“Añadir Subdominio”**.



Esto nos llevará a una pestaña lateral la cual nos pedirá los datos del subdominio:

Nombre del subdominio * .

Introduzca * para crear un subdominio wildcard.

Configuración de hosting

Raíz del documento *

Ruta al directorio principal del sitio web.

* Campos obligatorios

Hay que tener en cuenta que el proyecto debe apuntar a la **carpeta public**, por ejemplo:

/httpdocs/AHFDWESDemoScan/public

INSTALACIÓN MÓDULO LARAVEL

Una vez creado el subdominio debemos de instalar el módulo de Laravel en el mismo, para ello nos dirigimos de nuevo al menú lateral, esta vez a la pestaña que dice Laravel, una vez allí haremos clic en el botón que dice “**Instalar la aplicación**”.

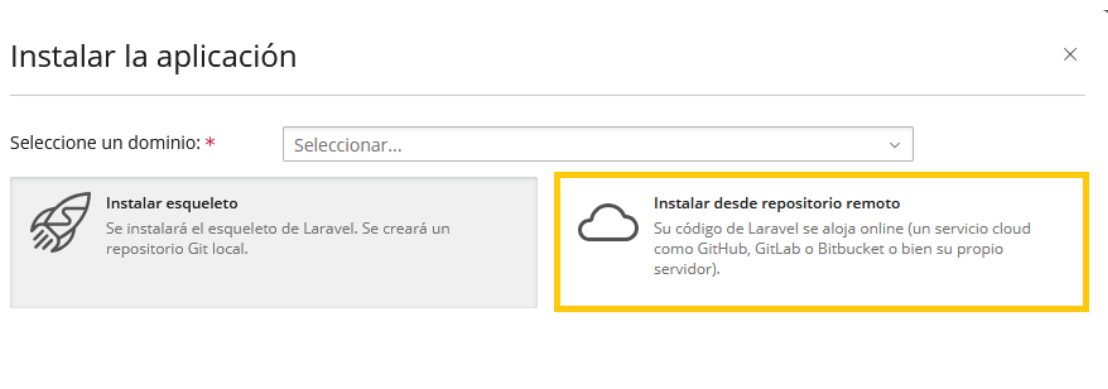
The image shows a control panel interface. On the left is a dark sidebar with a light blue header 'Sitios web y dominios'. Below it are menu items: 'Correo', 'Archivos', 'Bases de datos', 'Estadísticas', 'Usuarios', 'Cuenta', and 'Laravel'. The 'Laravel' item is highlighted with a yellow box. To the right of the sidebar, the main content area shows 'Elementos en total: 1'. Below this, there are two buttons: 'Instalar la aplicación' (highlighted with a yellow box) and 'Analizar'. Under the buttons, there is a 'Domain' label followed by the text 'AHFDWESDemoScan.alejandrohuefer.ieslossauces.es'.

Una vez hagamos clic se nos abrirá un menú lateral en el cual debemos indicar en que subdominio queremos instalar la aplicación.

SUBIR APLICACIÓN A EXPLOTACIÓN

También nos dará la opción a elegir de que forma queremos instalar la aplicación, si solamente el esqueleto o queremos subirla desde un repositorio de Github.

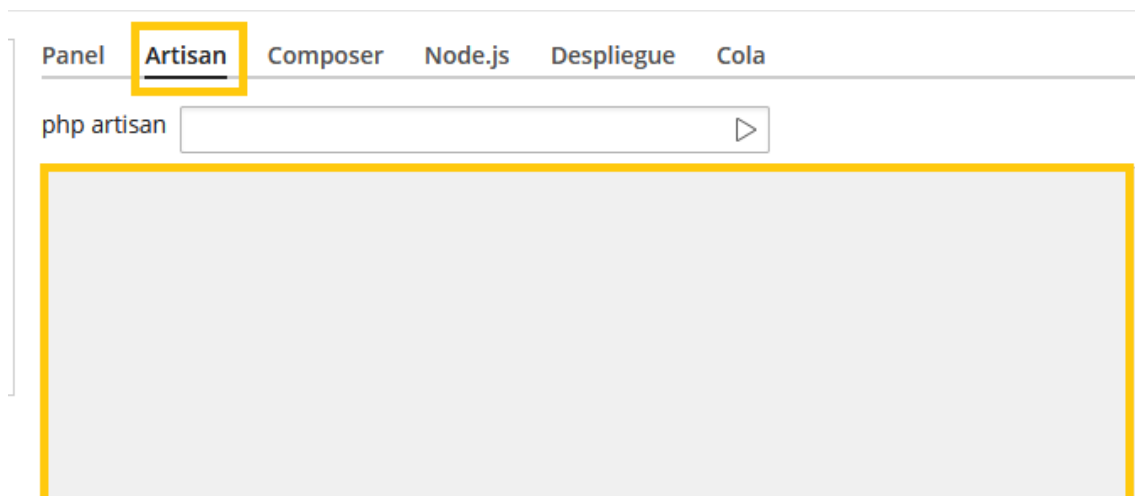
Nosotros elegiremos la opción del **repositorio de Github** y le proporcionaremos el enlace de la **rama máster**.



Esto nos subirá el proyecto a nuestro entorno de explotación.

CONFIGURACIÓN PROYECTO EN EXPLOTACIÓN

Una vez subida la reléase de nuestro proyecto a explotación debemos realizar la configuración de este, para ello lo primero que demos hacer será dirigirnos a Laravel ToolKit que ya viene integrado con el módulo instalado anteriormente.

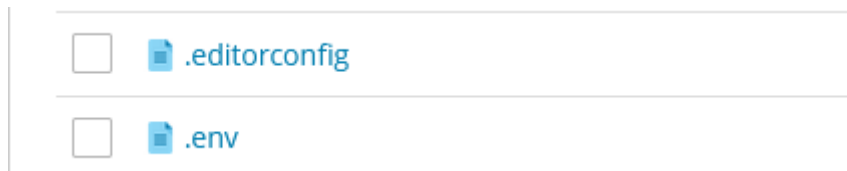


Una vez allí como hemos visto en la imagen anterior debemos de elegir la consola de Artisan y ejecutar el siguiente comando:

`cp .env.example .env`

Esto nos realizara la creación del archivo `.env` copiando el contenido del archivo `.env.example`.

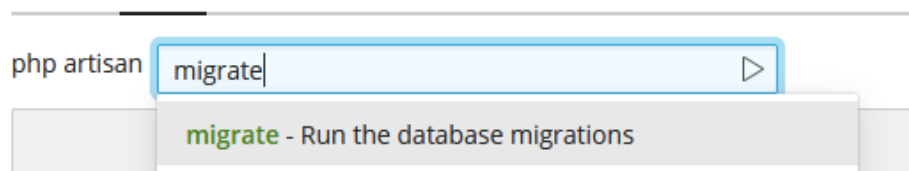
Podemos comprobarlo en los archivos de nuestro proyecto:



Este archivo debemos abrirlo y configurarlo con los diferentes datos de nuestra aplicación en explotación, así como los datos de conexión con la base de datos.

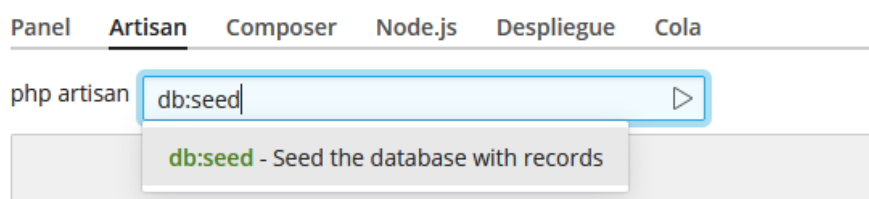
EJECUTAR MIGRACIONES

Por último, en la consola de Artisan de nuevo debemos ejecutar el siguiente comando, para ejecutar las migraciones y que se creen las tablas en la base de datos creada anteriormente:



Como podemos comprobar la consola es bastante intuitiva a la hora de completar.

Este mismo procedimiento debemos hacerlo con los **seeders** con el siguiente comando, también en esa misma consola:



Una vez realizada todas estas acciones nuestra aplicación estará desplegada correctamente en explotación y podremos comprobarlo y acceder a ella mediante la URL del subdominio.

WEBGRÁFIAS Y REFERENCIAS

RECURSOS Y BIBLIOGRAFÍA

El siguiente apartado incluye todas las fuentes de información o herramientas utilizadas para la elaboración de este proyecto, así como las menciones a todas las personas involucradas en él.

Documentación Oficial Laravel: <https://laravel.com/docs/12.x/>

Librería DemoParser para extracción de información:

<https://github.com/LaihoE/demoparser.git>

Foro de Reddit para Laravel: <https://www.reddit.com/r/laravel/>

Uso de la Inteligencia Artificial Claude: <https://claude.ai/onboarding>

CONCLUSIONES

Tras finalizar el proyecto y todo su estudio en el siguiente apartado vamos a comentar las conclusiones a las que me ha llevado el desarrollo de este, así como los nuevos conceptos aprendidos o el trabajo por realizar a futuro.

- La realización de este proyecto me ha llevado a entender y mejorar mi conocimiento con el framework Laravel de forma más avanzada.
- He aprendido en la utilización de node JS para procesar código JavaScript fuera del navegador web.
- He cogido conocimiento sobre los archivos demos así como a explotarlos mediante una librería externa y analizar posteriormente la información que almacenan.
- Se ha realizado una aplicación base en Laravel como MVP (Producto Mínimo Viable) de este proyecto.

MOTIVO DE LA ELECCIÓN DEL PROYECTO

El principal motivo de la elección del proyecto es mi amor por los deportes en general desde que era pequeño, la capacidad de análisis del deporte convencional siempre me ha entusiasmado y los deportes electrónicos también (Deportes los cuales se realizan en diferentes videojuegos ya sea de manera online o presencial).

Debido a estas dos pasiones decidí hace un año aproximadamente investigar sobre el nivel de análisis en los deportes electrónicos dándome cuenta de que, a pesar de ser ejecutados con las mejores tecnologías, en el aspecto de extracción de datos y análisis estaban muy por debajo del nivel que presentaban deporte como el fútbol o el baloncesto profesional debido a esto y a la poca información pública sobre este aspecto decidí investigar y realizar este proyecto.

UTILIDAD DEL PROYECTO

Es una aplicación la cual dota a los equipos profesionales de Counter Strike de la funcionalidad de extraer estadísticas de aquellas partidas que disputen sus jugadores para proceder con un análisis y poder mejorar su rendimiento con los datos como apoyo.

MÓDULOS DEL CICLO RELACIONADOS

Tras en desarrollo de esta aplicación web se han visto implementados varios módulos impartidos anteriormente en el ciclo.

- **Desarrollo Web Entorno Servidor:** Se ha desarrollado la aplicación del lado del servidor mediante el framework Laravel, incluyendo aquí el desarrollo de los modelos, controladores o de la base de datos.
- **Desarrollo web Entorno Cliente:** La utilización del lenguaje del lado del cliente JavaScript, aunque en nuestro caso lo ejecutásemos del lado del servidor, es un lenguaje impartido en dicha asignatura durante el ciclo.
- **Despliegue de aplicaciones web:** Se ha desplegado el desarrollo de la aplicación en explotación tras su desarrollo, así como la utilización del sistema de control de versiones git para el control de cambios realizados en el proyecto.
- **Diseño de Interfaces:** Se ha utilizado el framework Tailwind CSS como hoja de estilos de las vistas Blade.

LINEAS DE CONTINUIDAD A FUTURO

En cuanto a los planes de futuro del desarrollo de esta aplicación web se busca optimizar el flujo de trabajo de esta, así como la implementación de nuevas funciones mediante la utilización de nuevas técnicas de desarrollo o extracción y guardado de datos.

Por último, se realizará el estudio e implementación de un web service como fuente de datos de la propia aplicación con usuarios externos.

Con el desarrollo de web service se busca poder implementar los datos extraídos en plataformas externas de análisis estadísticos proporcionando así a los usuarios una fuente de datos fiable y segura.

Se buscará implementar la visualización de la posición de los jugadores en el mapa dentro de la propia aplicación permitiendo a los usuarios avanzar y retroceder en una línea de tiempo representativa de la duración de la partida.

AGRADECIMIENTOS

El desarrollo de este proyecto no hubiera sido posible sin la ayuda y colaboración de las personas citadas a continuación:

Tutor del Proyecto: Heraclio Borbujo.

Tutora de Prácticas: Hermelinda Ramos.

Instructor de prácticas: Jorge García.

Jefe de prácticas: David González.